Hazard-Aware Lunar Rover Navigation and Simulation Using MATLAB

A Research-Level Image Processing, Terrain Risk Modeling, and A Path Planning Framework*

Ayush Bhatt

Department of Aerospace / Space Systems Engineering

Research Manuscript Draft

Abstract

Autonomous lunar surface navigation requires a rover to interpret uncertain terrain, avoid hazards, and generate a traversable route between mission points. This paper presents a MATLAB R2025a-compatible simulation framework for hazard-aware lunar rover navigation. The system creates or loads a lunar surface image, applies manual image preprocessing, models craters and boulders, constructs roughness, slope, shadow, and hazard-clearance maps, and then generates a rover cost map for grid-based path planning. A* search is implemented from scratch and compared with Dijkstra search to evaluate planning efficiency. Unlike an earlier crater-to-crater shortest-path prototype, the proposed method treats craters as obstacles and unsafe regions rather than as waypoints, allowing the rover to navigate through safer terrain corridors. The final simulation generated a cratered lunar scene with 69 detected or modeled craters, produced a 1707.0-pixel route, achieved 0.00% hard-hazard crossing, and expanded 14,156 A* nodes compared with 14,644 Dijkstra nodes. The results demonstrate how image-derived terrain risk maps can support autonomous lunar navigation research, mission visualization, and educational prototyping.

Keywords: lunar rover, path planning, A* search, Dijkstra algorithm, hazard detection, crater detection, terrain cost map, MATLAB simulation, autonomous navigation

Paper Structure

1. Introduction

Lunar exploration is moving toward sustained surface operations, where robotic systems must travel through irregular and poorly illuminated terrain. The lunar surface contains impact craters, crater rims, ejecta, boulders, rough regolith, and steep local slopes. For rover mobility, these features are not only visually important but operationally critical because they affect traction, stability, localization, energy consumption, and mission safety.

The objective of this research is to build a computational pipeline that transforms a lunar surface image into a mission-level navigation plan. The model is not limited to drawing a shortest path between detected crater centers. Instead, it interprets craters and terrain irregularities as hazards, converts them into a cost field, and uses path planning to produce a safer route from a landing site to a target site.

The project is implemented as a single MATLAB R2025a-compatible script. A key engineering decision was to avoid dependency on local helper functions and toolbox-only commands, making the model easier to run on different MATLAB installations. The script can use a real image named chandrayaan_image.jpg or generate a synthetic lunar terrain with craters, rocks, shadows, roughness, and elevation. It outputs visual evidence including preprocessing figures, crater feature plots, hazard masks, cost maps, 2D navigation overlays, 3D terrain reconstruction, route telemetry, and rover animation.

2. Related Work and Baseline Motivation

The earlier project version used a Python/OpenCV-style workflow: load a lunar image, apply Gaussian blur and normalization, convert to grayscale, detect edges, extract crater contours, build a graph between craters, and apply Dijkstra shortest-path search. That baseline demonstrated a useful image processing and graph theory connection, but it planned between crater locations rather than through safe traversable terrain. The upgraded system therefore changes the planning philosophy: craters become obstacles and risk regions, while free terrain becomes the navigation domain.

The motivation is consistent with modern lunar terrain-relative navigation and hazard detection research. NASA lunar mission planning emphasizes that craters, boulders, slopes, and low-angle illumination complicate surface mobility and crew or rover navigation [1]. Terrain-relative navigation and hazard-detection systems also depend on image processing, reference mapping, onboard perception, and guidance algorithms [2]. In this project, those ideas are converted into a compact MATLAB research prototype suitable for demonstration and future expansion.

2.1 Difference from the Previous Prototype

Aspect	Earlier Python Prototype	Proposed MATLAB Research Model
Terrain representation	Detected crater positions and proximity graph	Continuous hazard and cost maps
Crater interpretation	Crater centers acted as nodes	Crater interiors, rims, and inflated zones act as hazards
Path planning	Dijkstra shortest path between graph nodes	A* and Dijkstra comparison on terrain cost grid
Risk factors	Distance between detected contours	Roughness, slope, shadow, boulders, crater clearance
Output	Single shortest-path plot	Preprocessing, hazard model, cost model, 2D/3D route, telemetry, animation, CSV reports

3. Proposed System Architecture

The proposed framework consists of six major modules: terrain acquisition, preprocessing, crater and hazard modeling, terrain-cost construction, graph-based route planning, and mission visualization. The complete pipeline is designed to be understandable and reproducible, while still producing research-style figures and quantitative metrics.

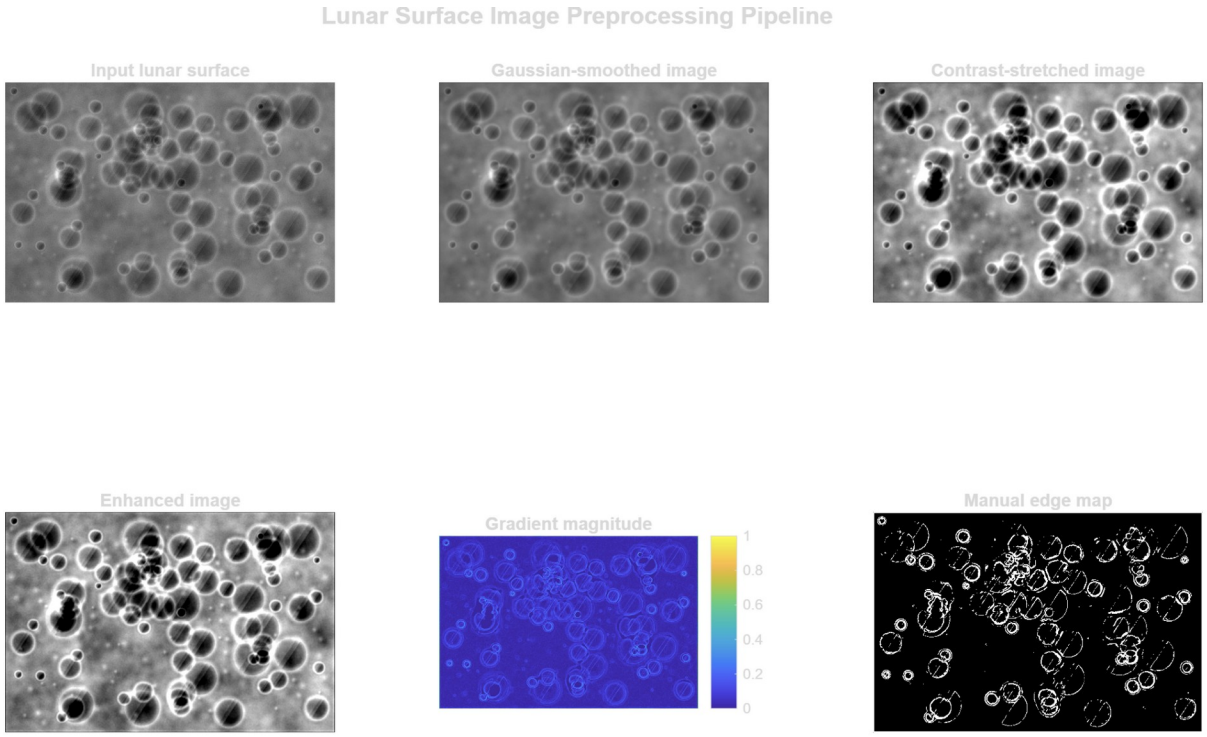

Figure 1. Lunar surface image preprocessing pipeline showing input terrain, smoothing, contrast stretching, enhancement, gradient magnitude, and manual edge detection.

3.1 Terrain Generation and Input Handling

The script first checks whether a real lunar image named chandrayaan_image.jpg exists. If present, the image is normalized and converted into grayscale. If no image is present, the script generates a synthetic lunar surface at 720 x 1080 pixels. The synthetic model uses multi-scale Gaussian-smoothed noise to create regolith texture, then adds craters, boulders, shadowed regions, an illumination gradient, and an approximate elevation field. The synthetic configuration starts with 82 craters and 120 rocks, after which overlapping crater candidates are filtered to produce the mission crater set.

3.2 Image Preprocessing

Preprocessing uses manual operations to avoid toolbox-specific dependency. The image is smoothed using a Gaussian kernel, contrast-stretched through percentile clipping, enhanced with unsharp masking, and converted into a gradient-magnitude edge map. These operations improve crater rim visibility and emphasize local terrain discontinuities.

$$G(x)=\exp(-x^2/(2\sigma^2)) / \sum \exp(-x^2/(2\sigma^2))$$

$$I_c = \text{clamp}((I_b - P_{1.5})/(P_{98.5} - P_{1.5}), 0, 1)$$

$$M_g = \sqrt{(dI/dx)^2 + (dI/dy)^2}$$

4. Mathematical Model and Algorithm Design

4.1 Crater Feature Extraction and Risk Ranking

Each crater is represented by a center, radius, area, rim gradient strength, inner-to-outer contrast, and risk score. In synthetic mode, crater centers and radii are known from the generation process; in real-image mode, a radial candidate search estimates likely crater positions. Duplicate or highly overlapping candidates are removed. Risk ranking is then computed as a weighted combination of rim strength, contrast, and normalized crater size.

$$Risk_i = 0.45\ R_i + 0.25\ C_i + 0.30\ (r_i / r_{max})$$

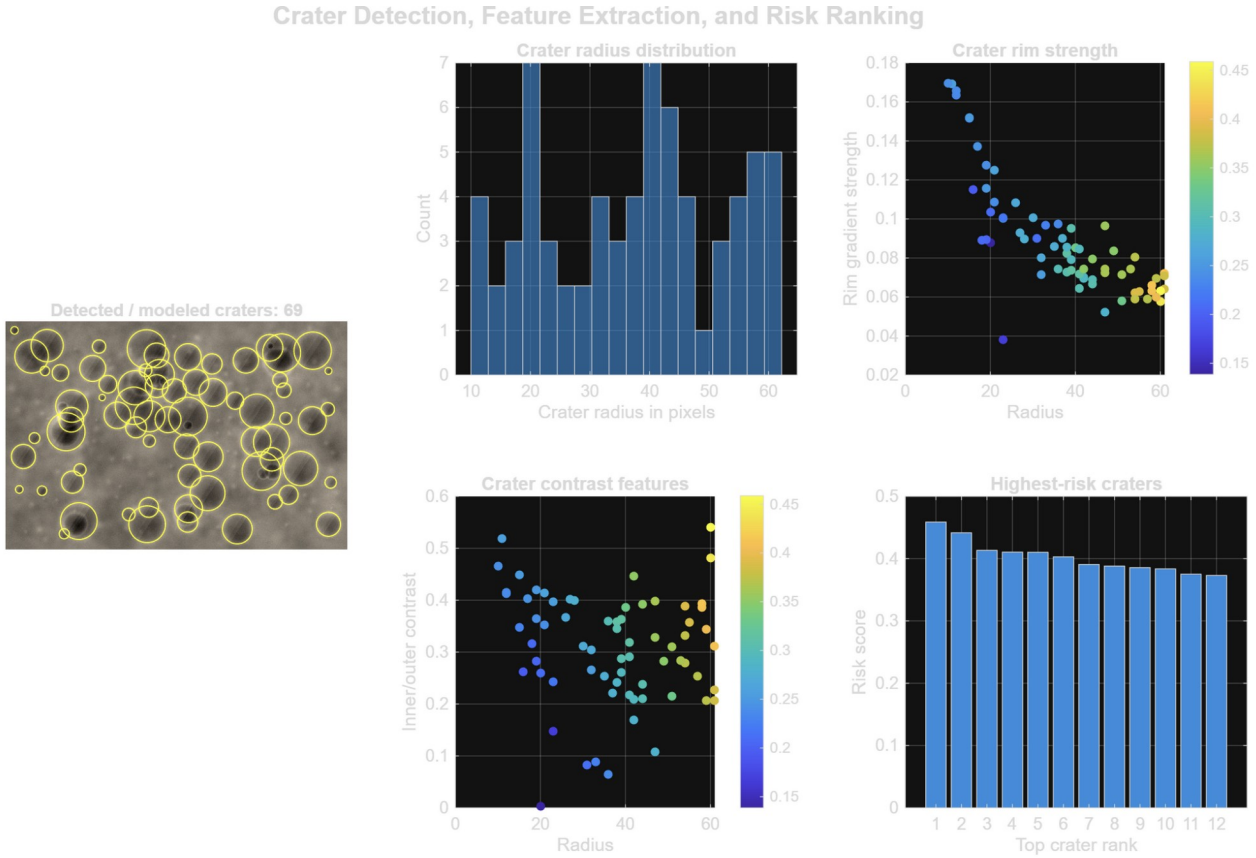

Figure 2. Crater detection, feature extraction, radius distribution, rim strength, contrast features, and highest-risk crater ranking.

4.2 Hazard Model

The hazard map is a binary representation of terrain cells that should not be crossed under normal navigation. It combines crater cores, crater rims, inflated crater safety margins, boulder regions, high-slope cells, and shadow-risk cells. The inflation step is important because a rover must avoid not only crater interiors but also unstable rim regions and nearby unsafe boundary zones.

$$H = H_{crater\_core} \cup H_{crater\_rim} \cup H_{boulder} \cup H_{slope} \cup H_{shadow}$$

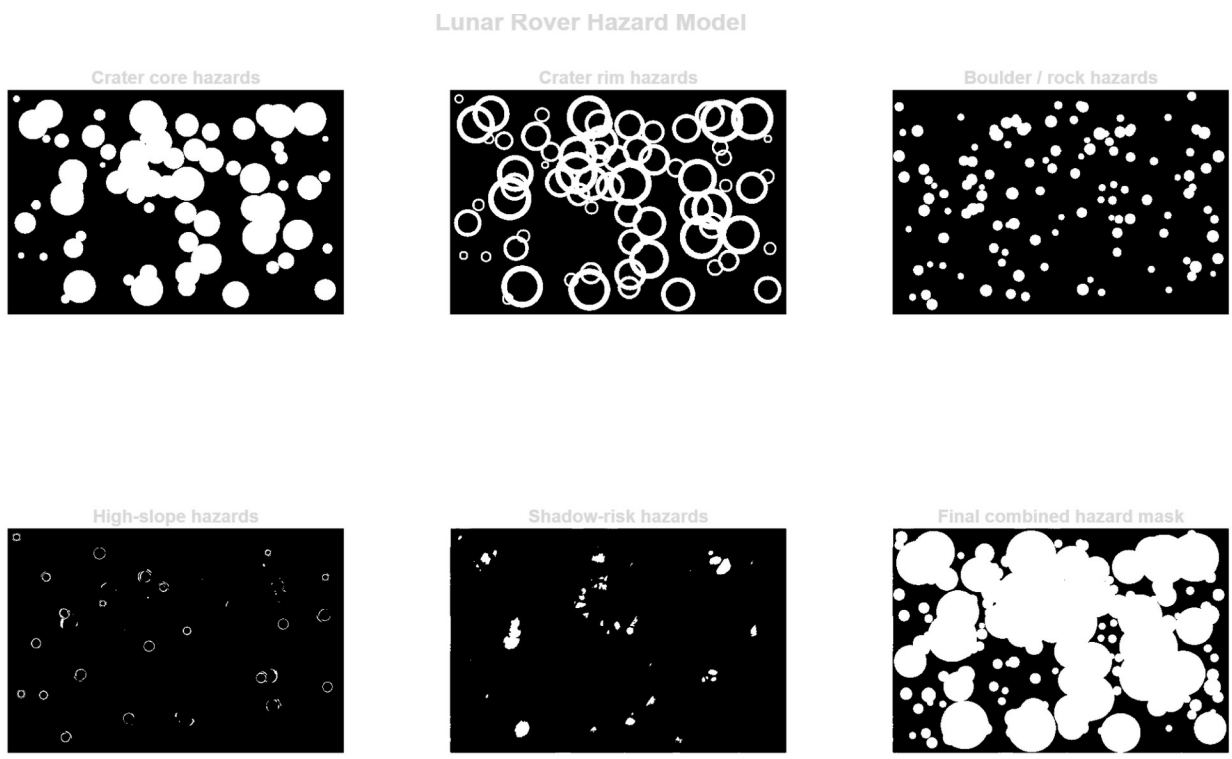

Figure 3. Hazard model components: crater core hazards, crater rim hazards, boulder hazards, high-slope hazards, shadow-risk hazards, and final combined hazard mask.

4.3 Terrain Cost Map

The navigation cost map is a continuous risk field. It assigns low cost to smooth, well-lit, hazard-distant terrain and high cost to rough, steep, shadowed, or near-hazard regions. Hard hazard cells are set to infinite cost for visualization and to a very large penalty for planning, which guarantees that a route can still be generated while strongly discouraging hazard traversal.

$$C = 1 + 7R + 5S + 2.2Q + 10 \exp(-D_h/18)$$

Here, R is roughness, S is slope, Q is shadow risk, and D_h is approximate distance from the nearest hazard. The distance map is computed using a manual chamfer-distance forward and backward pass, replacing toolbox functions while preserving the essential concept of hazard clearance.

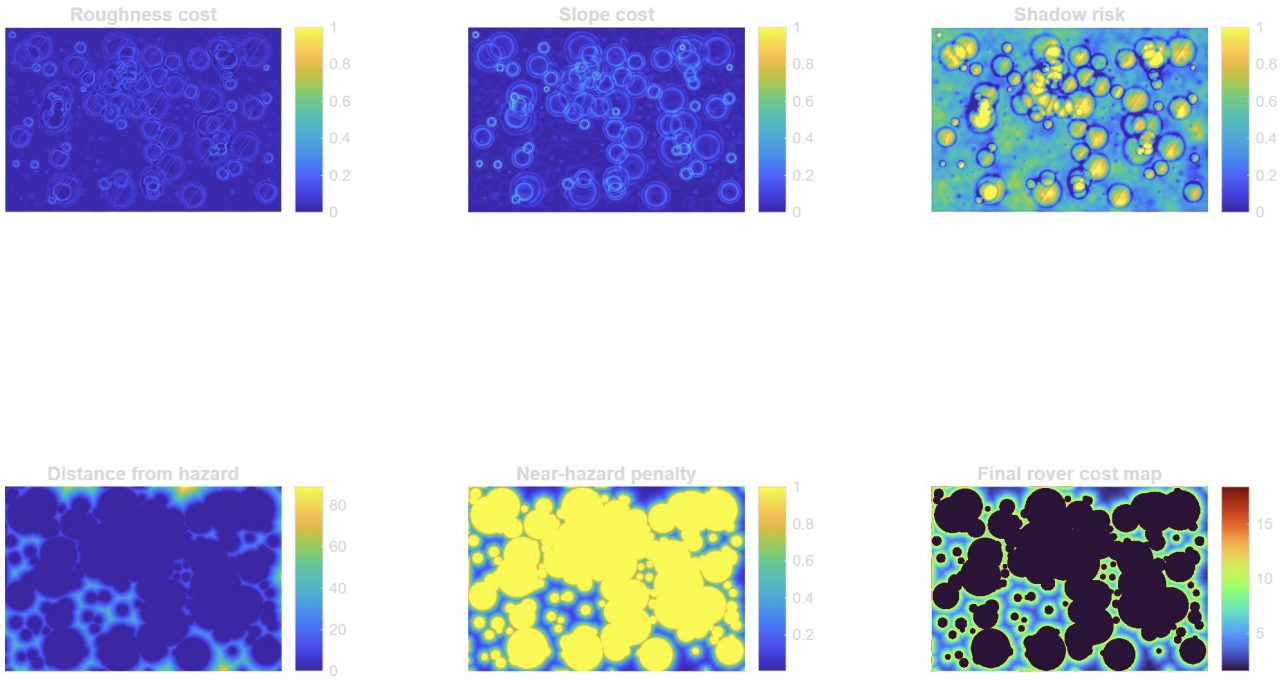

Figure 4. Cost-map components including roughness cost, slope cost, shadow risk, distance from hazard, near-hazard penalty, and final rover cost map.

4.4 A* and Dijkstra Search

The terrain cost map is downsampled into a planning grid with 8-connected motion. A* search is used as the primary planner because it combines accumulated path cost with a heuristic estimate of remaining distance. Dijkstra search is implemented as a comparison baseline by removing the heuristic term. Both planners use positive movement costs based on local terrain cost and Euclidean step distance.

$$f(n) = g(n) + h(n)$$

$$g(n\_next) = g(n) + d(n,n\_next) * (C\_n + C\_next)/2$$

With an admissible Euclidean heuristic, A* prioritizes nodes that are both low-cost and directionally useful toward the goal. Dijkstra expands outward from the start more uniformly, which makes it a strong correctness baseline but often less efficient for single-source, single-target navigation.

5. Experimental Setup

The experiment was executed in synthetic lunar terrain mode because no external image was required. The generated scene was processed through the entire pipeline and evaluated using mission statistics recorded in the dashboard and animation outputs. The landing point was placed near the lower-left portion of the map and the target point near the upper-right portion, forcing the rover to cross a crater-rich environment.

Parameter	Value Used in Experiment
Scene resolution	720 x 1080 pixels
Initial synthetic craters	82
Synthetic rocks/boulders	120
Final detected/modeled craters	69
Planning grid step	5 pixels

Roughness weight	7.0
Slope weight	5.0
Shadow risk weight	2.2
Near-hazard weight	10.0
Crater inflation multiplier	1.25
A* maximum iterations	250,000
Dijkstra maximum iterations	250,000

6. Results and Analysis

6.1 Final Route and Hazard Avoidance

The final 2D overlay shows the rover route avoiding the majority of crater interiors and inflated hazard zones. The red route follows narrow safer corridors near the image boundaries and across available terrain gaps. The green landing marker and blue target marker show a complete start-to-goal traversal.

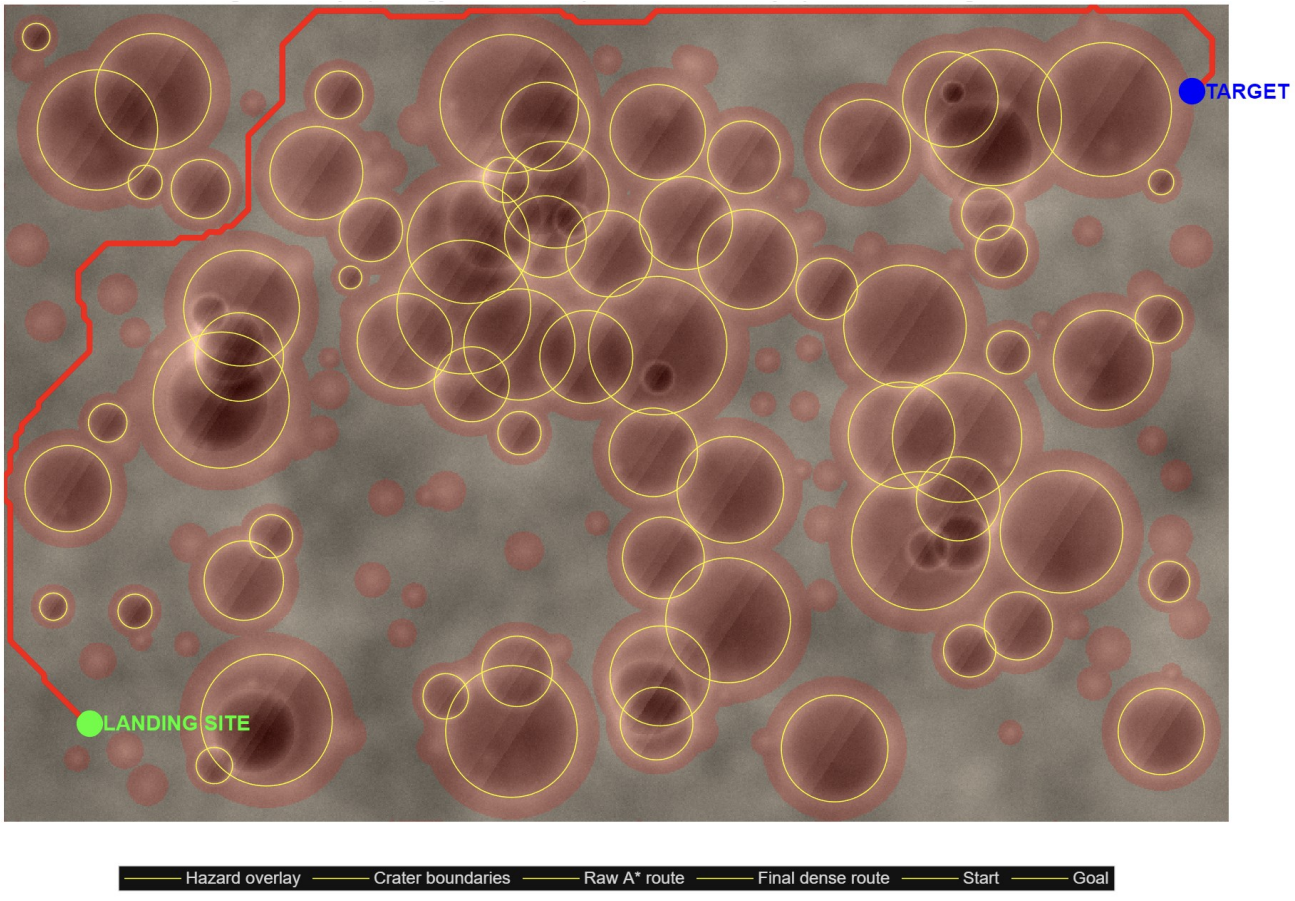

Figure 5. Final 2D hazard-aware rover route with crater boundaries, red hazard overlay, landing site, target site, raw A route, and dense final route.*

6.2 3D Terrain Reconstruction

The 3D visualization converts the image-derived elevation estimate into a surface plot. Although the elevation map is synthetic or image-derived rather than true lunar digital elevation data, it provides a visually useful approximation for studying how the planned route behaves across ridges, depressions, and cratered terrain.

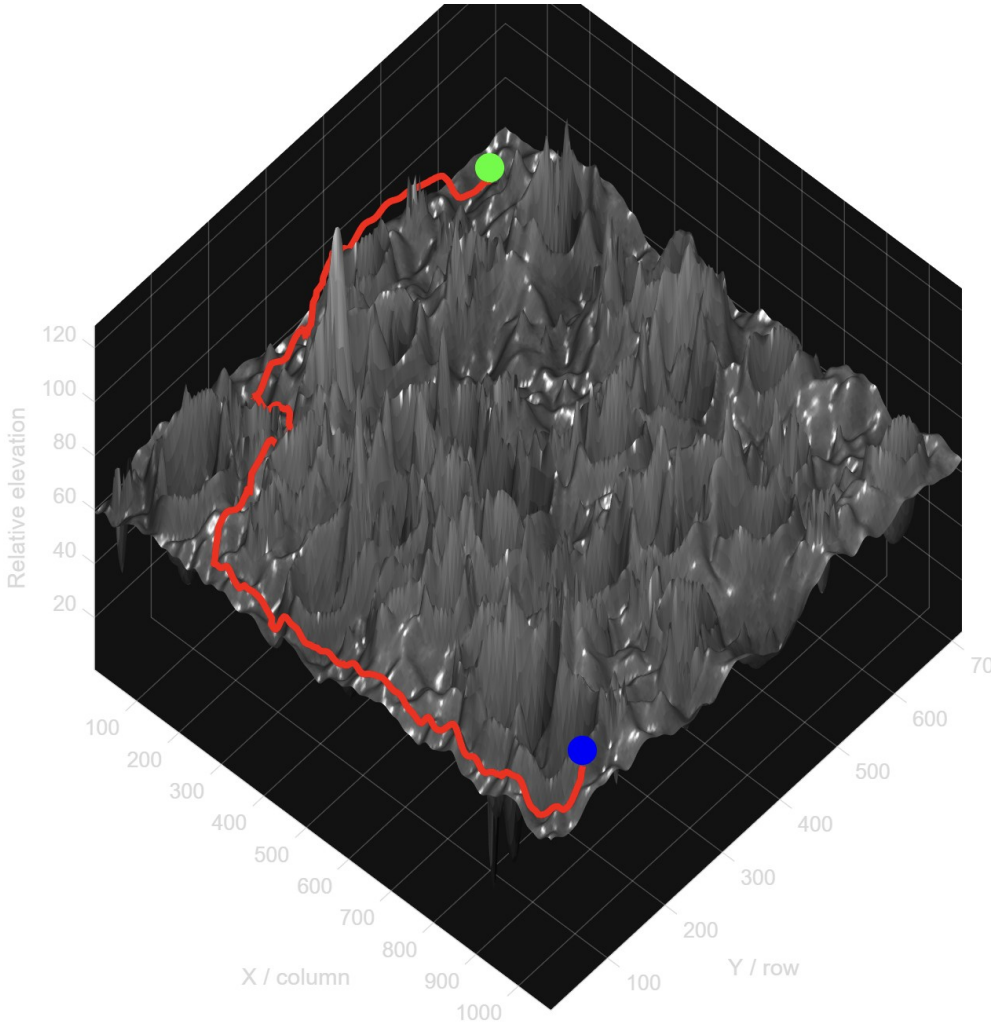

Figure 6. Three-dimensional lunar terrain reconstruction with planned rover route shown in red.

6.3 Mission Dashboard Metrics

The mission dashboard summarizes the quantitative performance of the simulation. The output route length was 1707.0 pixels and the computed A* cost was 2032.2. The estimated mission energy was 2200.6 units and the estimated traversal time was 94.8 seconds. The route maintained 0.00% hard-hazard crossing, indicating that the generated path did not directly enter hard hazard cells in the final result.

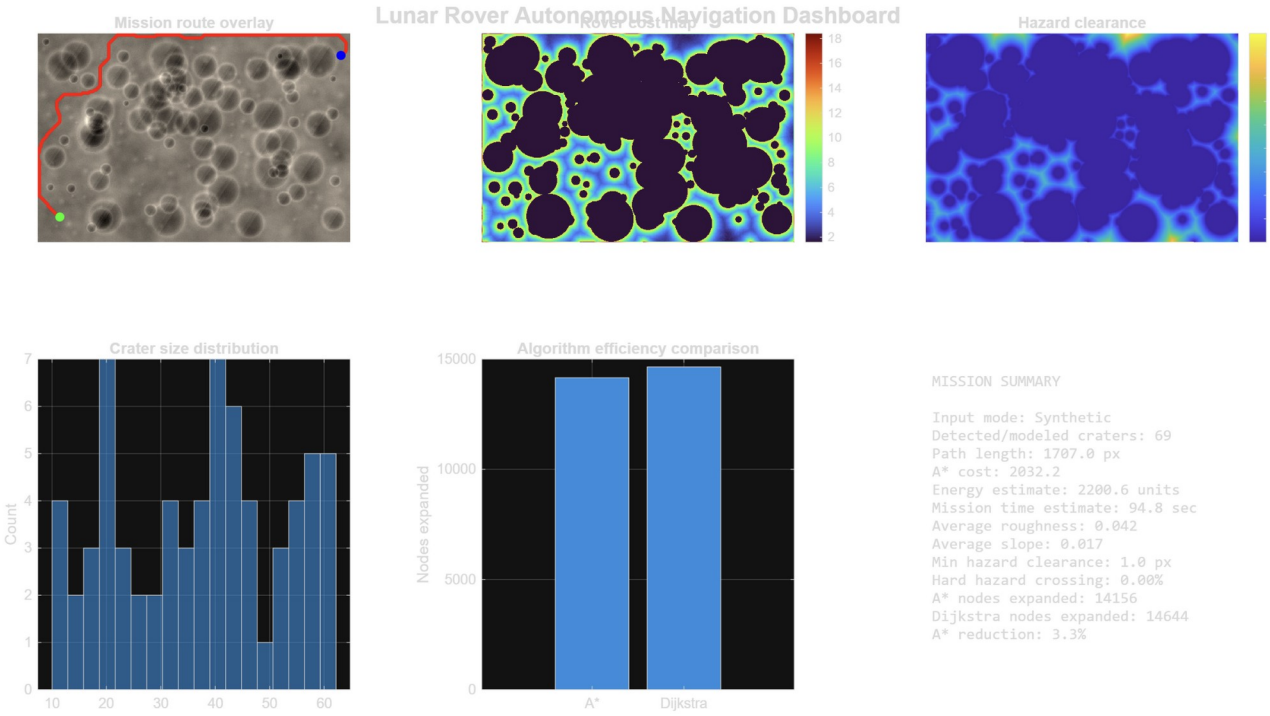

Figure 7. Mission dashboard showing route overlay, rover cost map, hazard clearance, crater-size distribution, algorithm-efficiency comparison, and mission summary.

Metric	Observed Result
Input mode	Synthetic lunar terrain
Detected/modeled craters	69
Path length	1707.0 pixels
A* route cost	2032.2
Estimated energy	2200.6 units
Estimated mission time	94.8 seconds
Average roughness	0.042
Average slope	0.017
Minimum hazard clearance	1.0 pixel
Hard hazard crossing	0.00%
A* nodes expanded	14,156
Dijkstra nodes expanded	14,644
A* node expansion reduction	3.3%

6.4 Route Telemetry

Route telemetry plots show how local cost, roughness, slope, and hazard clearance change as the rover progresses. Peaks in terrain cost occur where the rover moves near hazards or through more complex surface regions. The clearance plot is especially important because it shows when the route passes close to crater boundaries or boulder fields.

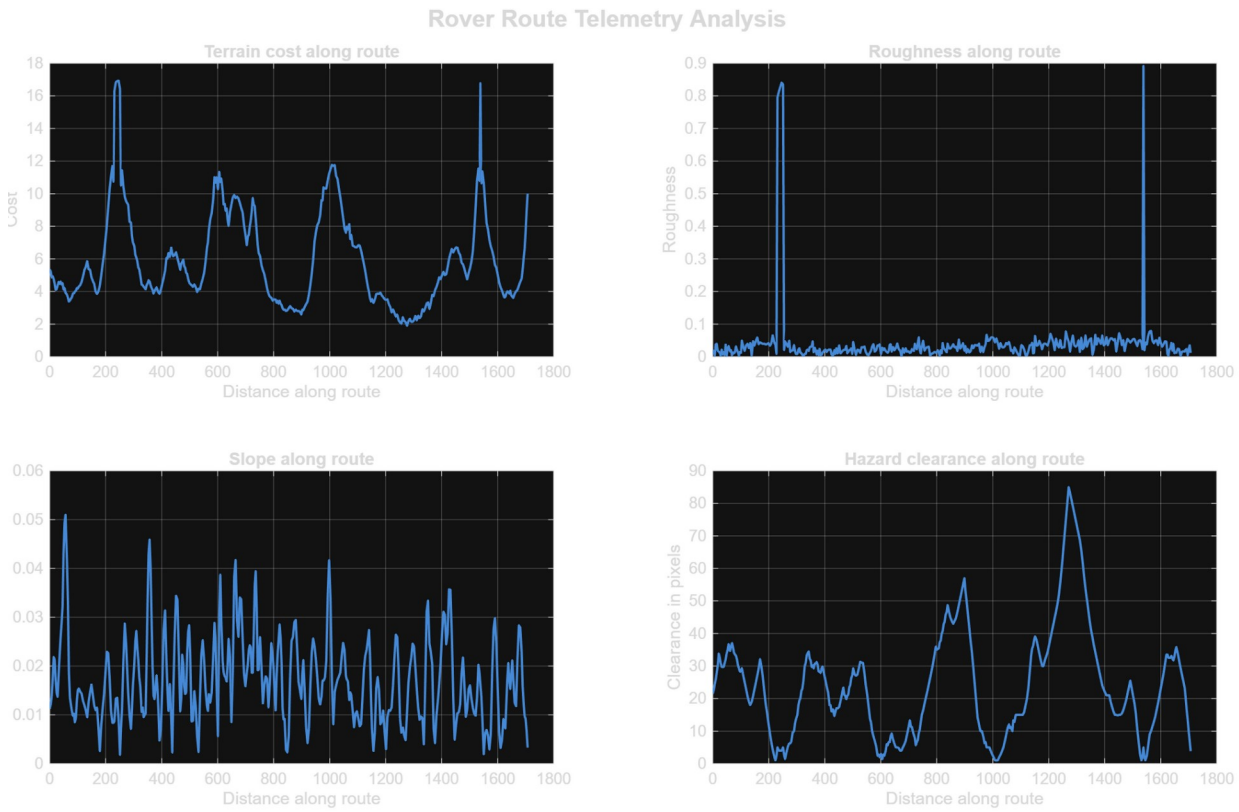

Figure 8. Rover route telemetry showing terrain cost, roughness, slope, and hazard clearance along the full route.

6.5 Animated Rover Simulation

The final animation frame demonstrates an autonomous rover traveling along the planned A* route. The rover telemetry includes waypoint count, traveled distance, battery estimate, local cost, roughness, slope, shadow risk, hazard clearance, hazard-state flag, and planner type. Near the end of the route, the displayed telemetry showed waypoint 381/384, traveled distance of 1655.7 pixels, battery estimate of 28.2%, local cost of 6.61, roughness of 0.005, slope of 0.010, shadow risk of 0.355, hazard clearance of 13.4 pixels, and inside-hazard flag equal to 0.

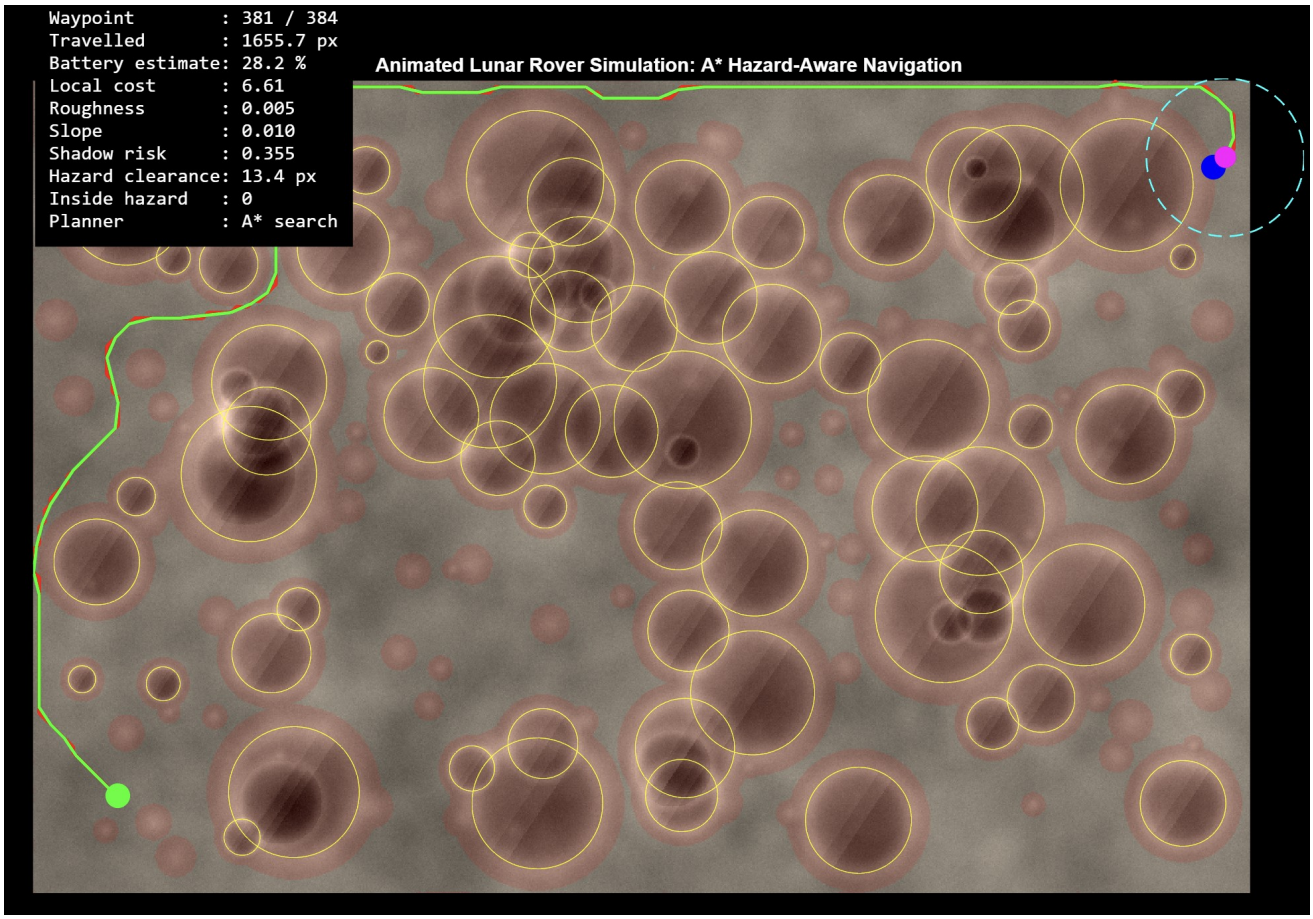

Figure 9. Animated lunar rover simulation final frame with live telemetry, hazard overlay, crater boundaries, sensor radius, route trail, and A planner label.*

7. Discussion

The results show that the upgraded system behaves more like a mobility-aware rover planner than a simple crater graph demo. The earlier approach connected crater centers using proximity edges, which is useful for explaining graph theory but unrealistic for rover navigation because crater centers are generally unsafe locations. The new method reverses that logic: crater interiors and rims become obstacles, and the rover searches through terrain that minimizes combined roughness, slope, shadow, and hazard-nearness penalties.

The A* improvement over Dijkstra in this specific run was 3.3% fewer node expansions. This reduction is modest because the map contains large hazard regions and boundary-following behavior, which limits how strongly the heuristic can focus the search. However, A* still provides a more mission-relevant planning structure because it directs expansion toward the target while respecting terrain cost. On larger maps or maps with more open safe corridors, the advantage of A* is expected to become more significant.

The telemetry plots add another research-level dimension because they make the route explainable. Instead of only showing a final path, the system reports how risk changes along the traversal. This makes it possible to identify critical segments, compare alternative routes, and justify why the rover chooses a longer but safer path.

8. Limitations and Future Work

- The elevation map is synthetic or image-derived and should not be interpreted as real digital elevation model data.
- Crater detection in real-image mode is approximate because the project intentionally avoids toolbox-only functions such as imfindcircles.

- The rover model is kinematic and does not simulate wheel slip, suspension dynamics, regolith sinkage, thermal limits, or communication constraints.
- The energy estimate is a relative engineering metric rather than a calibrated physical model.
- The route planner currently uses a 2D cost grid. Future work could include slope-constrained 3D planning, local replanning, SLAM, uncertainty maps, and multi-objective optimization.

Future improvements should integrate actual Lunar Reconnaissance Orbiter imagery or digital elevation data, validate crater detection against labeled datasets, add rover vehicle dynamics, compare A*, D*, RRT*, and PRM planners, and develop a multi-objective cost function including energy, risk, illumination, communication, and science value.

9. Conclusion

This research paper presented a complete MATLAB-based lunar rover navigation framework that transforms a lunar surface scene into a hazard-aware mission route. The system models craters, boulders, shadows, slopes, and rough regions, generates a cost map, applies A* path planning, compares against Dijkstra, and visualizes the mission through 2D overlays, 3D terrain, telemetry plots, dashboards, and animation. The final output demonstrated a 1707.0-pixel route through a crater-rich synthetic surface with 0.00% hard-hazard crossing. The project therefore provides a strong foundation for lunar mobility research, educational demonstrations, and future mission-planning prototypes.

References

[1] NASA. M2M-30044 Lunar Surface Data Book Baseline. NASA Technical Reports Server, 2025.

[2] Restrepo, C. I., et al. Building Lunar Maps for Terrain Relative Navigation and Hazard Detection. NASA Technical Reports Server / AIAA SciTech, 2022.

[3] Pedersen, L., et al. High Speed Lunar Navigation for Crewed and Remotely Piloted Vehicles. NASA Technical Reports Server, 2010.

[4] Hart, P. E., Nilsson, N. J., and Raphael, B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics, 1968.

[5] Dijkstra, E. W. A Note on Two Problems in Connexion with Graphs. Numerische Mathematik, 1, 269-271, 1959.

[6] Bhatt, A. lunar_rover_sim.m: MATLAB R2025a Single-File Hazard-Aware Lunar Rover Navigation Simulation, 2026.

[7] Presentation 2-2.pdf. Lunar Surface Images Stimulation and Visualisation: Using Python for Crater Detection and Shortest Path Calculation, 2024.

[8] Antriksh Hackathon.pdf. Python crater detection and Dijkstra shortest path baseline implementation, 2024.

Appendix A: Reproducibility Notes

- The simulation was implemented as a MATLAB script rather than a function file to reduce missing-helper-function errors during execution.
- The script avoids toolbox-only functions such as imfindcircles, edge, imshow, imgaussfilt, imresize, bwdist, strel, imdilate, and adapthisteq, relying instead on base operations such as conv2, gradient, imagesc, surf, plot, patch, and matrix indexing.
- If chandrayaan_image.jpg is not found, the script automatically generates a synthetic lunar terrain, making the experiment reproducible without external image dependencies.
- The output folder contains saved figures, CSV reports for crater features and path waypoints, and a project summary file.